

使用多資料庫持續性功能建構智慧型電子商務目錄

來源：<https://codelabs.developers.google.com/next26/polyglot-cross-cloud-retail?hl=zh-tw>

步驟數：12

使用 AlloyDB、MongoDB、Cloud Storage、BigQuery 和 MCP Toolbox 建構智慧型電子商務目錄。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 設定 Cloud Storage](#)
4. [4. 設定 AlloyDB](#)
5. [5. 在 Google Cloud 中設定 MongoDB Atlas](#)
6. [6. 設定 BigQuery](#)
7. [7. 瞭解應用程式端對端流程](#)
8. [8. 設定 MCP Toolbox 並部署至 Cloud Run](#)
9. [9. 設定電子商務應用程式並部署至 Cloud Run](#)
10. [10. 瞭解應用程式](#)
11. [11. 清除所用資源](#)
12. [12. 恭喜](#)

1. 簡介

在現代零售業中，資料是多元且廣泛的生態系統。您擁有穩固的交易資料 (價格和庫存)、「雜亂」的多型目錄 (電子產品規格與服裝尺寸)，以及 PB 級的行為記錄。強迫將這些內容整合到單一巨石中，不僅會造成技術債，還會破壞使用者體驗。

在本程式碼研究室中，您將設計出可協調下列項目的多語言強大系統：

- [AlloyDB](#)：交易主幹，可提供高速一致性和圖片嵌入。
- [Google Cloud 上的 MongoDB Atlas](#)：提供彈性且不限結構定義的目錄層。
- [Cloud Storage](#)：即時趨勢預測的分析中樞。
- [BigQuery](#)：高解析度數位倉庫。

「獨門配方」是什麼？您將使用 [MCP Toolbox for Databases](#)，以智慧方式自動調度管理並整合在 Cloud Run 上執行的資料來源，做為語意橋樑，然後使用 [Agent Development Kit \(ADK\)](#) 部署多代理聊天應用程式。您不只是建構搜尋列，而是建構智慧零售大腦，瞭解情境、遵守限制，並彌合原始資料與人類意圖之間的差距。

The Impossible User Query

標準電子商務代理程式無法進行多維度推理 (結合負面限制、視覺相似度和即時庫存)。舉例來說，我通常會想與這類零售網站對話：

「嘿，我正在規劃高海拔攝影之旅，請推薦幾款風格類似「AeroGlow Pro」的防潑水背包，但不要有任何皮革零件。此外，請告訴我這些產品是否實際有現貨，以及其他攝影師是否曾在評論中抱怨背帶耐用度不佳。」

為什麼這個查詢是「代理程式殺手」：

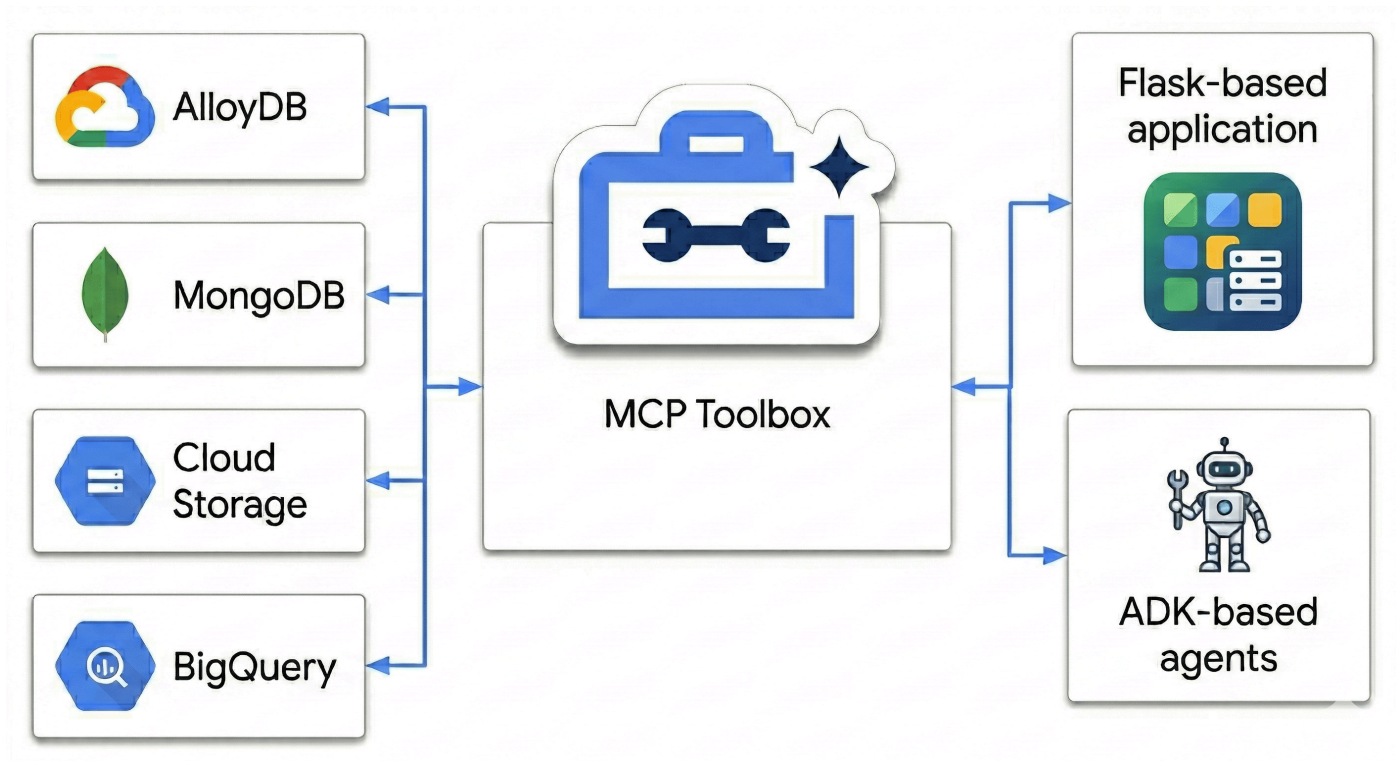
- **視覺相似度 (AlloyDB + 向量搜尋)**：「風格與 AeroGlow Pro 相似」需要比較圖片嵌入項目。
- **負向限制 (MongoDB)**：「不含任何皮革」需要透過彈性的巢狀屬性進行篩選，這類屬性通常不在標準 SQL 結構定義中。
- **即時庫存 (AlloyDB)**：「實際有現貨」需要即時交易檢查 (而非過時的搜尋索引)。
- **語意合成 (BigQuery + 多重代理程式)**：如要分析「錶帶耐用度」的評論，代理程式必須即時彙整 BigQuery 中非結構化的意見回饋。

大多數零售業機器人只會看到「背包」和「皮革」，然後顯示 10 個皮革背包。我們如何阻止這種情況？

因為我們不只是比對關鍵字，我們使用 MCP Toolbox，讓代理程式同時「推論」AlloyDB 中的交易事實和 MongoDB 中的彈性屬性。讓我們一起打造。

學習內容

- 為核心產品資料佈建 [AlloyDB](#) 叢集
- 設定 [Google Cloud 上的 MongoDB Atlas](#)，儲存半結構化產品詳細資料
- 建立 [Cloud Storage](#) bucket，用於提供產品圖片
- 將 [MCP Toolbox for Databases](#) 部署到 [Cloud Run](#)，確保資料存取方式一致
- 執行 ETL 程序，將資料推送至 [BigQuery](#) 進行分析
- 以自然語言與 AI 代理對話。



必要條件

- 網路瀏覽器，例如 [Chrome](#)
 - 已啟用計費功能的 Google Cloud 雲端專案
 - 免費的 [Google Cloud 上的 MongoDB Atlas](#) 帳戶
-

步驟 2 · 約 5 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用必要的 API

執行下列指令，啟用所有必要的 API：

```
gcloud services enable \  
  alloydb.googleapis.com \  
  bigquery.googleapis.com \  
  storage.googleapis.com \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  iam.googleapis.com \  
  secretmanager.googleapis.com \  
  compute.googleapis.com \  
  servicenetworking.googleapis.com \  
  aiplatform.googleapis.com
```

3. 設定 Cloud Storage

Cloud Storage 可做為非結構化媒體素材資源 (例如產品圖片) 的大型儲存空間。

1. 前往 Google Cloud 控制台的「Cloud Storage」，然後點選「建立值區」。
2. 為 bucket 指定全域不重複的名稱 (例如 `ecommerce-app-images`)。
3. 點選「建立」。
4. 如要允許範例應用程式存取圖片，而不需驗證，請取消勾選「強制禁止公開存取這個 bucket」選項，然後按一下「確認」。
5. 前往「權限」分頁標籤。
6. 在「Permissions」(權限) 中，按一下「Grant access」(授予存取權)。
7. 在「New principals」(新增主體) 中輸入 `allUsers`。
8. 在「請選擇角色」選單中，依序選取「Cloud Storage」>「Storage 物件使用者」。
9. 按一下「儲存」，然後按一下「允許公開存取」，確認要將資源設為公開。

上傳預留位置圖片

[BRK2-149-multidb-ecommerce](#) 使用預留位置圖片，提供最佳視覺體驗。

1. 在 Cloud Shell 中，複製 `next-26-sessions` 存放區：

```
git clone https://github.com/GoogleCloudPlatform/next-26-sessions.git
```

2. 前往 `UploadImages` 資料夾：

```
cd next-26-sessions/BRK2-149-multidb-ecommerce/UploadImages
```

3. 在 Google Cloud 控制台中，前往「Cloud Storage」，然後點選「Buckets」(值區)。
 4. 按一下新建立的 bucket 名稱。
 5. 依序點選「上傳」>「上傳檔案」，選取下載的範例圖片，然後按一下「開啟」。
-

4. 設定 AlloyDB

AlloyDB 是結構化、交易和重要資料 (例如產品 ID、名稱、SKU、價格和庫存) 的單一資料可靠來源。此外，AlloyDB 也為 AI 代理提供相似度搜尋功能，可進行推薦和自然語言查詢。

如要使用網頁介面，為開發環境快速設定叢集、執行個體和網路，請按照本[程式碼研究室](#)中的步驟操作。

佈建 AlloyDB 叢集

1. 前往 Google Cloud 控制台的「AlloyDB for PostgreSQL」。
2. 點選「建立叢集」。
3. 在「叢集 ID」部分輸入 `ecommerce-cluster`。
4. 為 `postgres` 使用者設定高強度密碼。如要學習，可以使用 `alloydb`。
5. 保留「Database Version」(資料庫版本) 的預設值。
6. 在「Region」(區域)，選取 `us-central1` (或偏好的區域)。

設定主要執行個體

1. 在「Instance ID」(執行個體 ID) 中輸入 `ecommerce-cluster-primary`。
2. 在「可用區可用性」部分，選取「單一可用區」。
3. 在「機型」中，選擇小型機型 (例如 N2、4 個 vCPU、32 GB RAM)。
4. 在「私人 IP 連線」中，選取「私人服務連線 (PSA)」，然後選取 `default` 網路。如果尚未設定預設網路，請按一下「確認網路設定」建立網路。
5. 在「Public IP Connectivity」中，選取「Enable Public IP」核取方塊，讓 MCP 工具箱在本程式碼研究室中正常連線。
6. 在「已授權的外部網路」中，輸入 `0.0.0.0/0`。選取「我瞭解風險」核取方塊，然後按一下「儲存」。
7. 點選「建立叢集」。

注意：請務必記下公開 IP 位址 (類似 `34.124.240.26`)。

建立叢集和執行個體最多需要 10 分鐘。

初始化資料庫

1. 按一下左側導覽選單中的「AlloyDB Studio」。
2. 在「資料庫」下拉式選單中，選取 `postgres`。
3. 選取「內建驗證」即可登入資料庫。
4. 在「Username」(使用者名稱) 中使用 `postgres` 使用者。
5. 在「Password」(密碼) 部分輸入您先前設定的密碼。
6. 按一下「Authenticate」(驗證)。
7. 在編輯器檢視畫面中，開啟新的「未命名的查詢」分頁。
8. 複製下列 DDL，然後點選「執行」：


```
CREATE TABLE products_core_table (  
  product_id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  sku VARCHAR(50) UNIQUE NOT NULL,  
  price NUMERIC(10, 2) NOT NULL,  
  stock INT NOT NULL  
);
```

9. 在 Cloud Shell 中，前往 `BRK2-149-multidb-ecommerce` 資料夾：

```
cd next-26-sessions/BRK2-149-multidb-ecommerce
```

10. 在 Cloud Shell 中開啟 `alloydb_insert_queries.sql` 檔案，然後複製插入查詢。

```
cat alloydb_insert_queries.sql
```

11. 在新的「未命名的查詢」分頁中，只貼上 `INSERT` 陳述式，然後按一下「執行」。

12. 在新的未命名查詢分頁中，複製下列 DDL 並點按「執行」，在 `products_core_table` 資料表上建立索引：

```
CREATE INDEX idx_products_core_sku ON products_core_table(sku);
```

建立圖片嵌入，供 AI 代理程式擷取類似產品

AI 代理程式整合功能會使用圖片嵌入來擷取類似產品。系統會使用 `multimodalembdding@001` 模型生成嵌入，並儲存在 AlloyDB 資料庫中。嵌入是 1408 維度的向量，並儲存在 `img_embeddings` 欄中。

我們必須先授予 AlloyDB 服務帳戶必要角色，才能存取 Cloud Storage，進而產生嵌入內容。

將角色授予 AlloyDB 服務帳戶，以存取 Cloud Storage

我們將「Storage 物件使用者」和「Storage 物件檢視者」角色授予 AlloyDB 服務帳戶，讓該帳戶能夠從 Cloud Storage bucket 讀取物件。

1. 前往「IAM 與管理」。
2. 按一下「授予存取權」。
3. 在「新增主體」欄位中，搜尋 AlloyDB 服務帳戶。服務帳戶類似於 `service-991742412753@gcp-sa-alloydb.iam.gserviceaccount.com`。
4. 按一下「選擇角色」。
5. 尋找並選取「Storage 物件使用者」角色。
6. 按一下「新增其他角色」，然後選取「Storage 物件檢視者」角色。
7. 按一下「新增其他角色」，然後選取「Vertex AI 使用者」角色。
8. 按一下 [儲存]。

啟用擴充功能

我們會使用 `pgvector` 和 `google_ml_integration` 擴充功能建構這個應用程式。`pgvector` 擴充功能可讓您儲存及搜尋向量嵌入。`google_ml_integration` 擴充功能提供多項函式，可讓您存取 Vertex AI 預測端點，並在 SQL 中取得預測結果。執行下列 DDL，啟用這些擴充功能：

1. 前往 Google Cloud 控制台的「AlloyDB for PostgreSQL」。
2. 按一下左側導覽選單中的「AlloyDB Studio」。
3. 在編輯器檢視畫面中，開啟新的「未命名的查詢」分頁。
4. 複製下列 DDL，然後點選「執行」：

```
CREATE EXTENSION IF NOT EXISTS vector;  
CREATE EXTENSION IF NOT EXISTS google_ml_integration;
```

使用嵌入初始化資料庫

1. 在 `products_core_table` 中新增 `img_embeddings` 欄。

```
ALTER TABLE products_core_table  
ADD COLUMN img_embeddings vector(1408);
```

2. 為圖片生成嵌入，並儲存在 `img_embeddings` 欄中。

```
UPDATE products_core_table  
SET img_embeddings = google_ml.image_embedding(  
    model_id => 'multimodalembembedding@001',  
    image => 'gs://<STORAGE_BUCKET_NAME>/' || sku || '.jpg',  
    mimetype => 'image/jpeg')  
WHERE sku IN (  
    SELECT  
        sku  
    FROM  
        products_core_table  
    WHERE  
        img_embeddings IS NULL  
        AND sku IS NOT NULL  
    LIMIT 10  
);
```

將 `<STORAGE_BUCKET_NAME>` 替換為 Cloud Storage 值區名稱。

3. 由於 Studio 有 5 分鐘的限制，請至少重複先前的查詢 5 次，為整個集合生成圖片嵌入。如果這項查詢逾時，請將 `LIMIT` 變更為 `5`，然後重新執行查詢十次。這個步驟可能需要幾分鐘才能完成。

5. 在 Google Cloud 中設定 MongoDB Atlas

MongoDB 會儲存豐富的半結構化產品詳細資料，以及彈性的使用者行為資料 (例如點擊和瀏覽)。

建立 MongoDB 叢集

1. 前往 [Google Cloud 上的 MongoDB Atlas](#)，然後選取免費層級帳戶。
2. 選取「Free」叢集層級，然後輸入叢集名稱，例如 `ecommerce-cluster`。
3. 選取「Google Cloud」做為供應商，並確認區域與 Google Cloud 區域一致 (例如 `us-central1`)。
4. 按一下「建立部署作業」。
5. 按一下 [關閉]。

設定網路存取權

1. 在 Atlas 控制台中，前往「Database & Network Access」(資料庫和網路存取權)。
2. 按一下「IP 存取清單」。
3. 按一下「新增 IP 位址」。
4. 新增 `0.0.0.0/0`，允許從任何位置存取。
5. 按一下「確認」。

使用 `0.0.0.0/0` 可透過任何 IP 位址存取。這適用於臨時展示，但在實際工作環境中，您應限制特定 IP 範圍的存取權。

建立資料庫使用者

1. 在 Atlas 控制台中，前往「Database & Network Access」(資料庫和網路存取權)。
2. 按一下「資料庫使用者」。
3. 按一下「Add New Database User」(新增資料庫使用者)。
4. 選取「密碼」做為驗證方式。
5. 輸入使用者名稱 `store-user` 和密碼 `storeuser`。
6. 按一下「新增內建角色」，然後選取「讀取及寫入任何資料庫」。
7. 按一下「新增使用者」。

取得連線字串

1. 依序前往「資料庫」>「叢集」>「連線」。
2. 在「連結應用程式」中，按一下「驅動程式」。
3. 複製「Add your connection string into your application code」(將連線字串新增至應用程式程式碼) 中顯示的連線字串。字串看起來會像這樣：

```
mongodb+srv://store-user:<db_password>@ecommerce-cluster.g8vaekh.mongodb.net/?
appName=ecommerce-cluster
```

將 `db_password` 替換為 MongoDB 密碼。在本程式碼研究室中，這個值為 `storeuser`。

請儲存這個連線字串，稍後會用於 `MONGODB_CONNECTION_STRING` 環境變數。

建立資料庫和集合

1. 在 Atlas 控制台中，依序前往「Database」>「Clusters」>「Browse Collections」。
 2. 按一下「建立資料庫」，然後輸入詳細資料：
 - 資料庫名稱：`ecommerce_db`
 - 集合名稱：`product_details_collection`
 3. 按一下 [Create Database] (建立資料庫)。
 4. 在資料探索工具中，選取集合名稱。
 5. 按一下「新增資料」圖示，然後點選「插入文件」。
 6. 複製 `product_details_export.json` 中的 JSON 內容，然後貼到「插入文件」編輯器對話方塊。
 7. 按一下「插入」，插入文件陣列，並確認已新增 192 份文件。
 8. 在資料探索器中，按一下 `ecommerce_db` 資料庫旁的「建立集合 (+)」。
 9. 輸入 `user_interactions_collection` 做為集合名稱，然後按一下「建立集合」。
 10. 在資料探索工具中，選取 `user_interactions_collection` 集合。
 11. 按一下「新增資料」圖示，然後點選「插入文件」。
 12. 複製 `user_interactions_export.json` 中的 JSON 內容，然後貼到「插入文件」編輯器對話方塊。
 13. 按一下「插入文件」。
-

6. 設定 BigQuery

BigQuery 會彙整及分析過往的使用者行為，產生智慧報表和建議。

建立資料集

1. 前往 Google Cloud 控制台的「BigQuery」**BigQuery**。
2. 在「Explorer」窗格中，點選專案 ID 旁的三點選單，然後選取「建立資料集」。
3. 在「Dataset ID」(資料集 ID) 中輸入 `ecommerce_analytics`。
4. 點選「建立資料集」。

建立 Analytics 資料表

1. 在 BigQuery 工作區中開啟新查詢。
2. 執行下列 SQL 陳述式，建立將使用者連結至產品互動的摘要資料表：

```
CREATE TABLE ecommerce_analytics.user_product_interactions (  
  user_id STRING DEFAULT 'any user',  
  product_id STRING,  
  interaction_score INT  
);
```

將角色授予 MCP Toolbox 的 Compute 服務帳戶

我們會將角色授予用於工具箱的 Compute 服務帳戶。這麼做是為了讓 MCP Toolbox 存取 BigQuery、Secret Manager 和其他雲端服務。

如要找出專案編號，請前往 Google Cloud 控制台，依序點選「Cloud 總覽」>「資訊主頁」。記下「專案資訊」資訊卡中列出的專案編號。

如要授予角色，請完成下列步驟：

1. 前往「IAM 與管理」。
2. 按一下「授予存取權」。
3. 在「新增主體」欄位中，輸入名為 `YOUR_PROJECT_NUMBER-compute@developer.gserviceaccount.com` 的預設 Compute 服務帳戶。將 `YOUR_PROJECT_NUMBER` 替換為您的 Google Cloud 專案編號。
4. 按一下「選擇角色」。
5. 找出並選取「BigQuery 資料編輯者」角色。
6. 點選「新增其他角色」，然後選取「BigQuery 工作使用者」角色。
7. 按一下「Add another role」(新增其他角色)，然後選取「Secret Manager Secret Accessor」(Secret Manager 密鑰存取者) 角色。
8. 按一下「新增其他角色」，然後選取「編輯者」角色。
9. 按一下 [儲存]。

7. 瞭解應用程式端對端流程

為瞭解各元件如何相互運作，我們將建立使用多個資料庫和服務的簡易電子商務應用程式。這個應用程式採用 Python (Flask) 後端建構，並整合多項 Google Cloud 服務和資料庫。

瞭解目錄結構

在下一節中，您將複製 `BRK2-149-multidb-ecommerce` 存放區，並使用該存放區在本機執行應用程式。在本機測試應用程式後，我們會將 MCP Toolbox 和應用程式部署至 Cloud Run。

在這個目錄中探索下載的檔案。以下是高階目錄：

- `UploadImages`：儲存圖片資產，主要用於電子商務產品目錄的說明文件或視覺內容。
- `static`：儲存應用程式的靜態網頁素材資源，例如 CSS 和 JavaScript 檔案，用於設定使用者介面的樣式及新增互動功能 (來源)。
- `templates`：儲存 Python 應用程式使用的 HTML 範本 (Flask 可能是 Jinja2)，用於動態轉譯電子商務目錄的網頁 (來源)。
- `toolbox-implementation`：儲存 Model Context Protocol (MCP) Toolbox 的設定和實作詳細資料，使用預先定義的工具簡化多重資料庫的互動。

這個存放區中的檔案會共同運作，建構、設定及部署多資料庫電子商務應用程式。`app.py` 等中央檔案會整合 SQL 和 JSON 檔案中定義的各種資料來源，藉此協調後端作業，而設定檔則可確保順利部署至雲端環境：

- `app.py`：協調 Flask 後端和多個資料庫的整合。
- `agentengine.py`：初始化及設定 Vertex AI 代理的核心邏輯。
- `.env`：儲存資料庫和儲存空間連線的密鑰。
- `tools.yaml`：設定 MCP Toolbox，以執行多個資料庫作業。
- `Dockerfile`：定義容器映像檔和環境設定。
- `requirements.txt`：列出應用程式執行階段所需的 Python 程式庫。
- `tools.yaml`：MCP Toolbox 的設定。
- `Procfile`：指定部署作業的正式環境執行指令。
- `alloydb_insert_queries.sql`：包含關聯式資料的 SQL 查詢。
- `product_details_export.json` 和 `user_interactions_export.json`：提供 NoSQL 資料庫的 JSON 資料範例。
- `README.md`：引導您完成設定、部署及瞭解專案。

應用程式的端對端流程

- **AlloyDB 設定**：佈建高效能叢集，並使用提供的 SQL 指令碼建立 `products_core_table`，其中包含圖片嵌入的向量欄。
- **設定 MongoDB Atlas**：在 Google Cloud 上部署叢集，將產品屬性儲存在 `product_details` 中，並記錄 `user_interactions` 中的即時點擊串流。
- **BigQuery Analytics**：建立資料集來彙整互動記錄，以便執行複雜的分析查詢，從數百萬個事件中找出「前 5 大」熱門項目。
- **Cloud Storage 存放區**：建立公開 bucket 來存放高解析度產品圖片，並確保每個資產都能透過已簽署或公開的網址供前端存取。

- **部署 MCP Toolbox**：將 Toolbox 部署到 Cloud Run，建立中央 RESTful 橋接器，將自然語言意圖轉換為多資料庫查詢。
- **tools.yaml 設定**：定義「工具」，例如 `get_product_core_data` 或 `get_top_5_views`，將特定 SQL 和 NoSQL 作業對應至簡單且代理程式可讀取的名稱。
- **Flask 後端邏輯**：實作與 MCP Toolbox 介接的 `app.py` 路由，處理資料擷取作業的協調，並做為 UI 的 API。
- **多代理自動化調度管理**：在程式碼中設定 ADK 代理，推斷使用者意圖，並選取合適的「工具」，解決複雜的多來源零售查詢。
- **前端整合**：建構 `index.html` 介面，其中包含產品目錄、互動記錄功能、可瞭解產品成效分析的「Analytics」分頁，以及使用 ADK 多代理聊天室的專屬「代理」分頁，提供流暢的對話式購物體驗。

現在來實作自動調度管理和部署作業。

8. 設定 MCP Toolbox 並部署至 Cloud Run

MCP Toolbox 會抽象化多個資料來源，讓應用程式統一擷取及寫入資料。

在本機安裝 MCP Toolbox

1. 在 Cloud Shell 中，前往 `toolbox-implementation` 資料夾：

```
cd next-26-sessions/BRK2-149-multidb-ecommerce/toolbox-implementation
```

2. 下載 MCP Toolbox 二進位檔並設為可執行：

```
export VERSION=0.29.0
curl -L -o toolbox https://storage.googleapis.com/genai-
toolbox/v$VERSION/linux/amd64/toolbox
chmod +x toolbox
```

設定 tools.yaml

您需要定義 AlloyDB、MongoDB 和 BigQuery 的抽象概念。`tools.yaml` 檔案會告知 MCP 工具箱如何互動。

1. 使用內嵌編輯器建立及編輯 `tools.yaml` 檔案：

```
cloudshell edit tools.yaml
```

您可以在 [GitHub 存放區](#) 中找到完整的 `tools.yaml` 檔案。將內容複製到新的 `tools.yaml` 檔案。

2. 更新主機、使用者、密碼、專案 ID 和連線字串，與您在先前步驟中佈建的基礎架構相符：

資料庫	欄位	範例值
AlloyDB/BigQuery	<code>project_id</code>	<code>YOUR_PROJECT_ID</code>
AlloyDB	<code>region</code>	<code>us-central1</code>
AlloyDB	<code>cluster</code>	<code>ecommerce-cluster</code>
AlloyDB	<code>instance</code>	<code>ecommerce-cluster-primary</code>
AlloyDB	<code>database</code>	<code>postgres</code>
AlloyDB	<code>password</code>	<code>alloydb</code>

MongoDB	<code>connection_string</code>	<code>mongodb+srv://store-user:storeuser@ecommerce-cluster.urcxr6q.mongodb.net</code>
---------	--------------------------------	---

將角色授予 MCP Toolbox 的 Compute 服務帳戶

我們會將角色授予用於工具箱的 Compute 服務帳戶。這是為了讓 MCP Toolbox 存取 AlloyDB。

1. 前往「IAM 與管理」。
2. 按一下「授予存取權」。
3. 在「新增主體」欄位中，輸入名為 `YOUR_PROJECT_NUMBER-compute@developer.gserviceaccount.com` 的預設 Compute 服務帳戶。將 `YOUR_PROJECT_NUMBER` 替換為您的 Google Cloud 專案編號。

專案編號：如要找出專案編號，請前往 Google Cloud 控制台，依序點選「Cloud 總覽」>「資訊主頁」。記下「專案資訊」資訊卡中列出的專案編號。

4. 按一下「選擇角色」。
5. 找出並選取「BigQuery 資料編輯者」角色。
6. 按一下「新增其他角色」，然後選取「AlloyDB 用戶端」角色。
7. 按一下「新增其他角色」，然後選取「服務使用情形消費者」角色。
8. 按一下「新增其他角色」，然後選取「Storage 物件檢視者」角色。
9. 按一下 [儲存]。

測試工具 UI

1. 在 Cloud Shell 終端機中，在本機執行工具箱，提供 UI 服務：

```
./toolbox --ui
```

2. 在 Cloud Shell 中開啟通訊埠 5000 的網頁預覽，然後前往工具頁面。舉例來說，您可以透過以下網址查看工作階段網址：`https://5000-cs-71152278760-default.cs-asia-southeast1-cash.cloudshell.dev/ui`

在 Cloud Shell 中存取 MCP UI：Cloud Shell 會使用「網頁預覽」公開本機通訊埠。您必須使用網頁預覽網址並附加 `/ui`，例如：`https://5000-cs-.../ui?authuser=0`，而不是開啟 `localhost:8080`。否則只會顯示「Hello World」純文字訊息，而非載入的 UI 資訊主頁。

您會看到下列 MCP 工具箱使用者介面：

ask_alloydb_nl
execute_sql_tool
get_all_analytics
get_product_core_data
get_product_details
get_product_stats_by_category
get_top_5_views
get_total_interactions_count
insert_user_interaction
list_all_product_details

Tools

To inspect and test a tool, please click on one of your tools to the left.

What are Tools?

Tools define actions an agent can take, such as running a SQL statement or interacting with a source. You can define Tools as a map in the `tools` section of your `tools.yaml` file.

Some tools also use `parameters`. Parameters for each Tool will define what inputs the agent will need to provide to invoke them.

[Tools Documentation](#)

部署至 Cloud Run

將 MCP Toolbox 部署至 Cloud Run，做為安全代管服務，供應用程式查詢資料庫。我們會將設定儲存在 Secret Manager 中，保護私密的連線詳細資料。

1. 開啟新的 Cloud Shell 工作階段。
2. 前往 `toolbox-implementation` 資料夾：

```
cd next-26-sessions/BRK2-149-multidb-ecommerce/toolbox-implementation
```

3. 將 `tools.yaml` 設定上傳至 Google Secret Manager：

```
gcloud secrets create tools --data-file=tools.yaml
```

注意：如要為現有密鑰新增版本，請使用下列指令：

```
gcloud secrets versions add tools --data-file=tools.yaml
```

4. 使用公開的 MCP Toolbox 容器映像檔進行部署：

```
export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:0.29.0
export PROJECT_ID=$(gcloud config get-value project)

gcloud run deploy toolbox \
  --image $IMAGE \
  --region us-central1 \
  --service-account $(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")-compute@developer.gserviceaccount.com \
  --set-secrets "/app/tools.yaml=tools:latest" \
  --args="--tools-file=/app/tools.yaml","--address=0.0.0.0","--port=8080","--ui" \
  --allow-unauthenticated
```

5. 部署完成後，請記下系統提供的 Cloud Run 服務網址。畫面應該會類似這樣：`https://toolbox-*****-uc.a.run.app/ui`。

步驟 9 · 約 5 分鐘

9. 設定電子商務應用程式並部署至 Cloud Run

資料庫執行完畢，且 MCP Toolbox 抽象層部署完成後，我們就能執行 Flask 網頁應用程式！

如要提供產品目錄，Flask 應用程式會執行下列步驟來處理資料：

1. 擷取核心資料：從 AlloyDB 擷取完整產品清單 (`list_products_core`)。
2. 擷取詳細資料：從 MongoDB (`list_all_product_details`) 擷取所有產品詳細資料。
3. 合併清單：串連兩個清單。
4. 加入多媒體內容：為每個項目新增 Cloud Storage 圖片網址。

生成 Reasoning Engine 應用程式路徑

如要使用 Google Cloud 的 Vertex AI Reasoning Engine 初始化及註冊 AI 代理程式，請執行下列指令：

1. 在 Cloud Shell 終端機中，前往 `BRK2-149-multidb-ecommerce` 資料夾。

```
cd next-26-sessions/BRK2-149-multidb-ecommerce
```

2. 執行 `requirements.txt` 安裝依附元件

```
pip install -r requirements.txt
```

3. 執行 `agentengine.py` 指令碼，產生 Reasoning Engine 應用程式路徑：

```
python agentengine.py
```

畫面中會顯示如下的輸出結果：

```
projects/991742412753/locations/us-central1/reasoningEngines/4933254136889081856
```

設定環境變數

1. 建立並編輯 `.env` 檔案：

```
cloudshell edit .env
```

2. 將值替換為特定資料庫連線和新的 Cloud Run Toolbox 網址：

```
# 1. MongoDB Connection String
MONGODB_CONNECTION_STRING="mongodb+srv://<db_user>:<db_password>@cluster0.mongodb.net"

# 2. MCP Toolbox Server Location
# Must match the address where you run the toolbox server
MCP_TOOLBOX_SERVER_URL="https://toolbox-*****-uc.a.run.app"

# 3. Google Cloud Storage Bucket Name
GCS_PRODUCT_BUCKET="ecommerce-app-images"

# 4. Fallback image URL
FALLBACK_IMAGE_URL="https://storage.googleapis.com/ecommerce-media-bold-circuit-492711-n9/fallback.jpg"

# 5. Google Gen AI Vertex AI flag
GOOGLE_GENAI_USE_VERTEXAI=TRUE

# 6. Project ID
PROJECT_ID=codelab-project-491117

# 7. Google Cloud Location of AlloyDB, BigQuery databases
GOOGLE_CLOUD_LOCATION=us-central1

# 8. Reasoning engine application path
APP_NAME=projects/991742412753/locations/us-central1/reasoningEngines/4933254136889081856

# 9. Model ID
MODEL=gemini-1.5-flash-lite
```

您可能會發現並非所有圖片都會立即載入；根據屬性對應至複製應用程式資料集中的產品 SKU 的方式，部分連結可能會遺失或指向錯誤。請仔細檢查 GCS bucket 的公開存取政策，並確認檔案名稱對應正確。

將前端部署至 Cloud Run

1. 將網頁應用程式部署至 Cloud Run，完成架構：

```
gcloud run deploy polyglot --source . --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars \
  MONGODB_CONNECTION_STRING="<MONGODB_CONNECTION_STRING>", \
  MCP_TOOLBOX_SERVER_URL="<MCP_TOOLBOX_SERVER_URL>", \
  GCS_PRODUCT_BUCKET="<GCS_PRODUCT_BUCKET>", \
  FALLBACK_IMAGE_URL="<FALLBACK_IMAGE_URL>", \
  GOOGLE_GENAI_USE_VERTEXAI=TRUE, \
  PROJECT_ID="YOUR_PROJECT_ID", \
  GOOGLE_CLOUD_LOCATION=us-central1, \
  APP_NAME="<YOUR_REASONING_ENGINE_APP_PATH>", \
  MODEL="gemini-1.5-flash-lite"
```

替換下列值：

- `YOUR_PROJECT_ID`：您的 Google Cloud 專案 ID。
- `YOUR_REASONING_ENGINE_APP_PATH`：執行 `python agentengine.py` 的輸出內容，例如 `projects/991742412753/locations/us-central1/reasoningEngines/4933254136889081856`。
- `MCP_TOOLBOX_SERVER_URL`：MCP Toolbox 伺服器的網址，例如 `https://toolbox-*****-uc.a.run.app`。
- `GCS_PRODUCT_BUCKET`：Google Cloud Storage bucket 的名稱，例如 `ecommerce-app-images`。
- `MONGODB_CONNECTION_STRING`：MongoDB 資料庫的連線字串，例如 `mongodb+srv://store-user:storeuser@ecommerce-cluster.g8vaekh.mongodb.net`
- `FALLBACK_IMAGE_URL`：備用圖片的網址，例如 `https://storage.googleapis.com/ecommerce-app-images/fallback.jpg`

如果部署失敗，請執行 `gcloud auth login` 指令授權部署，然後再次執行部署指令。

您的應用程式現已上線！開啟 Cloud Run 提供的服務網址，即可查看 Multidb Ecommerce 目錄。網址會類似於 `https://polyglot-*****-uc.a.run.app/`。

10. 瞭解應用程式

- 1. 按一下「產品目錄」即可查看所有產品。



Polyglot Architecture Demo

🛒 Product Catalog

📊 ETL & Analytics

🤖 AI Agent



4-Person Camping Tent Deluxe

SKU: OUT-TN-4P-57 | Polyglot (AlloyDB + MongoDB)

\$299.00



4-Port GaN Fast Charger 100W

SKU: EL-CH-GN-19 | Polyglot (AlloyDB + MongoDB)

\$65.00

- 2. 按一下產品圖示即可查看產品詳細資料。你會發現圖片來自 Cloud Storage、產品詳細資料來自 MongoDB，而商品目錄來自 AlloyDB。



Cashmere Blend Crew Neck Sweater ×

\$110.00

Stock: 40

Hybrid Data Store

Cashmere Blend Crew Neck Sweater: An ultra-soft, lightweight sweater perfect for layering, featuring a refined crew neck and ribbed cuffs.

Care

Hand wash recommended

Knit

Fine Gauge

Material

70% Wool, 30% Cashmere

3. 與產品目錄互動，產生傳送至 **MongoDB** 的模擬檢視畫面和寫入內容。
4. 按一下「ETL 和 Analytics」即可查看產品分析。你會發現產品分析資料是從 BigQuery 擷取。

Data Pipeline

Run ETL Job

Success! Processed 6276 items.

Top Trending Products

#1 4-Person Camping Tent Deluxe

30 Views

#2 4-Port GaN Fast Charger 100W

20 Views

#3 Adjustable Height Standing Desk

12 Views

#4 Durable Gardening Tool Set (5pc)

11 Views

5. 按一下「AI 代理」分頁標籤，即可與 AI 代理互動。以自然語言提問，例如：

I'm planning a high-altitude photography trip.
Show me some weather-resistant backpacks similar in style to aero glow pro but without any leather components. Also, let me know if they are actually in stock and if other photographers have complained about the strap durability in the reviews.

 **Product Catalog**

 **ETL & Analytics**

 **AI Agent**

I'm planning a high-altitude photography trip. Show me some weather-resistant backpacks similar in style to aero glow pro but without any leather components. Also, let me know if they are actually in stock and if other photographers have complained about the strap durability in the reviews.

Here are some weather-resistant backpacks that fit your criteria:

Alpine Explorer Backpack 40L

- SKU: OUT-BP-AE-06 -
\$89.99

Show trending products

Check stock

Garden tools

Fitness gadgets

Ask about stock, trends, or product details...

Send

如您所見，搜尋結果完全符合我們的要求：不含皮革成分、有現貨，且評論中沒有關於背帶耐用度的負評。



Alpine Explorer Backpack 40L



\$89.99

Stock: 15

Hybrid Data Store

Alpine Explorer Backpack 40L: Rugged 40-liter backpack designed for long hikes and outdoor adventures.

Capacity	40L
Material	600D Nylon
Weight	1.2kg

11. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收費，請刪除本程式碼研究室建立的資源。

執行下列 Cloud Shell 指令：

```
gcloud run services delete toolbox --region us-central1 --quiet
gcloud run services delete multi-db-app --region us-central1 --quiet
bq rm -r -f -d $PROJECT_ID:ecommerce_analytics
gcloud storage rm --recursive gs://ecommerce-app-images
gcloud alloydb clusters delete ecommerce-cluster --region us-central1 --force --quiet
```

如要刪除整個 Google Cloud 雲端專案和所有資源，請執行下列指令 (選用)：

```
gcloud projects delete $PROJECT_ID
```

最後，開啟 Google Cloud 上的 MongoDB Atlas，並刪除您建立的 `cluster0` 叢集，確保不會有閒置的免費層級用量。

12. 恭喜

恭喜！您已成功建構跨雲端 Multidb 架構。

您已示範 MCP Toolbox 如何做為現代化專業應用程式的架構黏著劑。將正確的資料庫與正確的工作配對，可達成下列目標：

- 彈性資料寫入：適用於事件記錄的 MongoDB。
- 交易一致性：AlloyDB 可確保核心完整性。
- 高效能資料分析：BigQuery 適用於商業智慧。
- 統一開發：單一 Python 後端，使用 MCP Toolbox 抽象化所有複雜性。

參考文件

進一步瞭解相關的 Google Cloud 產品，並探索下列程式碼研究室：

- AlloyDB AI：[開始透過 AlloyDB AI 使用向量嵌入](#)
- [AlloyDB AI：AlloyDB 的多模態嵌入](#)
- MCP Toolbox：[在 AlloyDB 上安裝及設定 MCP Toolbox for Databases](#)

如要進一步瞭解本程式碼研究室中使用的產品，請參閱：

- [AlloyDB for PostgreSQL 說明文件](#)
 - [Google Cloud 上的 MongoDB Atlas 說明文件](#)
 - [BigQuery 說明文件](#)
 - [Cloud Storage 說明文件](#)
 - [Cloud Run 說明文件](#)
 - [Cloud Build 說明文件](#)
 - [導入 MCP Toolbox](#)
-